Introduction to Yale SOM HPC Server

Getting Started with Slurm and Spack

Connecting to the HPC

To access the HPC server, use SSH:

```
ssh <your-netid>@hpc.som.yale.edu
```

Once connected, you'll be on a login node. **Important:** Login nodes are for light tasks only (editing files, submitting jobs). All computational work must be submitted through the Slurm scheduler.

Introduction to Slurm

Slurm is a job scheduler that manages computational resources on the HPC cluster. Instead of running programs directly, you submit jobs that Slurm queues and executes when resources become available.

Basic Slurm Commands

Submitting a Job with `sbatch`

The primary way to run jobs is with `sbatch`, which submits a batch script:

```
sbatch my_job.sh
```

A basic job script (`my_job.sh`) looks like:

```
#!/bin/bash
#SBATCH --job-name=my_analysis
#SBATCH --output=output_%j.log
#SBATCH --error=error_%j.log
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G

# Your commands here
python my_script.py
```

Common `sbatch` options:

- `--job-name`: Name for your job

- `--output`: File for standard output (`%j` is replaced with job ID)
- `--error`: File for error messages
- `--time`: Maximum runtime (HH:MM:SS)
- `--ntasks`: Number of tasks (usually 1 for single-node jobs)
- `--cpus-per-task`: CPU cores to allocate
- `--mem`: Memory allocation (e.g., 8G, 16G)
- `--partition`: Queue/partition to use (ask your admin for available partitions)

Interactive Sessions with `srun`

For testing or interactive work, use `srun`:

```
srun --pty --cpus-per-task=2 --mem=4G --time=01:00:00 bash
```

This allocates resources and gives you an interactive shell on a compute node.

Monitoring Jobs

Check your job status:

```
squeue -u $USER
```

View detailed job information:

```
scontrol show job <job-id>
```

Cancel a job:

```
scancel <job-id>
```

View past jobs:

```
sacct
```

Loading Software with Spack

Spack is a package manager that provides access to scientific software. Instead of installing packages yourself, you load pre-compiled modules.

Basic Spack Commands

Finding Available Packages

Search for packages:

```
spack find
```

Search for specific software (e.g., Python):

```
spack find python
```

Loading Packages

Load a package to use it:

```
spack load python
```

Load a specific version:

```
spack load python@3.11
```

View currently loaded packages:

```
spack find --loaded
```

Unload a package:

```
spack unload python
```

Using Python

Loading Python

```
spack load python
```

Python Virtual Environments

It's recommended to use virtual environments for your projects:

```
# Load Python
spack load python

# Create a virtual environment
python -m venv ~/myproject_env

# Activate it
source ~/myproject_env/bin/activate

# Install packages
pip install numpy pandas matplotlib scikit-learn
```

Example Python Job Script

```
#!/bin/bash
#SBATCH --job-name=python_analysis
#SBATCH --output=results_%j.log
#SBATCH --time=02:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G

# Load Python
spack load python

# Activate virtual environment
source ~/myproject_env/bin/activate

# Run your script
python analysis.py
```

Using R

Loading R

```
spack load r
```

Installing R Packages

You can install R packages in your home directory:

```
spack load r
R

# In R console:
install.packages("tidyverse", repos="https://cloud.r-project.org")
install.packages("ggplot2", repos="https://cloud.r-project.org")
```

Example R Job Script

```bash
#!/bin/bash
#SBATCH --job-name=r_analysis
#SBATCH --output=results_%j.log
#SBATCH --time=02:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G

# Load R
spack load r

# Run your script
Rscript analysis.R
```

Workflow Example

A typical workflow looks like:

1. Connect to HPC: `ssh <netid>@hpc.som.yale.edu`
2. Load required software: `spack load python`
3. Prepare your scripts and data
4. Create a job submission script
5. Submit the job: `sbatch my_job.sh`
6. Monitor progress: `squeue -u $USER`
7. Review results in output files

Best Practices

- **Test interactively first:** Use `srun` to test your workflow before submitting large batch jobs
- **Estimate resources:** Request appropriate time and memory; jobs are killed if they exceed limits
- **Use job arrays:** For running the same script with different parameters, use Slurm job arrays
- **Check job efficiency:** Use `seff <job-id>` to see how much of your requested resources were actually used
- **Store data appropriately:** Use scratch space for temporary files and backed-up storage for important results

HPC Etiquette: Common Faux Pas to Avoid

The HPC is a shared resource. Following good etiquette ensures everyone can use the cluster effectively:

Don't Hog Resources

Issue: Requesting excessive resources (e.g., 64 CPUs, 256GB RAM) when your job only needs a fraction of that.

Impact: Your job sits in the queue longer, and other users are blocked from accessing resources they need.

Solution: Start with modest resource requests and scale up only if needed. Monitor your actual usage with `seff <job-id>` after jobs complete.

Don't Let Resources Sit Idle

Issue: Allocating resources and then not using them—especially interactive sessions left running overnight or jobs that finish early but still hold the allocation.

Impact: This is one of the worst violations of cluster etiquette. Resources sitting idle means other users' jobs are queued unnecessarily.

Solution:

- Always cancel interactive sessions when done: `exit` or Ctrl+D
- For batch jobs, ensure your time request is reasonable—slightly overestimating is fine, but don't request 24 hours for a 30-minute job
- Use `scancel <job-id>` to cancel jobs you no longer need

Request Collaborative Project Folders

Issue: Multiple team members working in scattered personal directories, making collaboration difficult and duplicating data.

Solution: For collaborative work, email the IT group to request a shared project folder. This provides:

- Centralized location for shared data and scripts
- Proper permissions for team access
- Better organization and reduced data duplication

How to request: Email the HPC IT team with your project name, team members, and storage needs.

Getting Help

- Check job status: `squeue -u $USER`
- View job details: `scontrol show job <job-id>`
- Check resource usage: `seff <job-id>`
- For system-specific information, contact your HPC administrator

Additional Resources

- Slurm documentation: https://slurm.schedmd.com/
- Spack documentation: https://spack.readthedocs.io/

Appendix: Example Files

Example Python Script: fibonacci.py

```python
#!/usr/bin/env python3
"""
Simple script to compute Fibonacci numbers.
Demonstrates basic Python script for HPC submission.
"""

import sys
import time

def fibonacci(n):
    """Compute the nth Fibonacci number."""
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b

    return b

def fibonacci_sequence(n):
    """Compute the first n Fibonacci numbers."""
    sequence = []
    for i in range(n):
        sequence.append(fibonacci(i))
    return sequence

if __name__ == "__main__":
    # Check if argument provided
    if len(sys.argv) < 2:
        print("Usage: python fibonacci.py <n>")
        print("Computing first 20 Fibonacci numbers by default...")
        n = 20
    else:
        n = int(sys.argv[1])

    print(f"Computing first {n} Fibonacci numbers...")
    start_time = time.time()

    # Compute the sequence
    sequence = fibonacci_sequence(n)

    elapsed_time = time.time() - start_time

    # Print results
    print(f"\nFirst {n} Fibonacci numbers:")
    for i, fib in enumerate(sequence):
        print(f"F({i}) = {fib}")

    print(f"\nComputation completed in {elapsed_time:.4f} seconds")
    print(f"The {n}th Fibonacci number is: {fibonacci(n)}")
```

Example Slurm Submission Script: submit_fibonacci.sh

```bash
#!/bin/bash
#SBATCH --job-name=fibonacci
#SBATCH --output=fibonacci_%j.log
#SBATCH --error=fibonacci_%j.err
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

# Print job information
echo "Job started at: $(date)"
echo "Running on node: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "========================================="

# Load Python from Spack
spack load python

# Run the Python script
# You can change the argument to compute more Fibonacci numbers
python fibonacci.py 30

echo "========================================="
echo "Job finished at: $(date)"
```

Using These Files

To use these example files:

1. Copy the Python script to a file named `fibonacci.py`
2. Copy the Slurm script to a file named `submit_fibonacci.sh`
3. Make the submission script executable: `chmod +x submit_fibonacci.sh`
4. Submit the job: `sbatch submit_fibonacci.sh`
5. Check job status: `squeue -u $USER`
6. View results: `cat fibonacci_<job-id>.log`